

SOCIALLY API Documentation

RESTful API for Social Media Platform

Base URL: `https://api.socialsphere.com/v1`

Authentication

- Method: JWT token + OTP verification
- Header Format:

```
Authorization: Bearer <your_jwt_token>
Content-Type: application/json
```

- OTP Flow:
 1. Request OTP via `/auth/login/request-otp`
 2. Verify OTP via `/auth/login/verify-otp` to receive JWT
-

Role-Based Access Control (RBAC)

Permissions are granted based on user roles:

Role	Permissions
user	Create posts, manage friend requests, like content, view public feeds
moderator	Delete posts, suspend users (moderation actions)
admin	Full system access (user management, configurations, and all moderation actions)

Endpoints

1. User Sign-Up

Register a new user

```
POST /auth/signup HTTP/1.1
Host: api.socialsphere.com
Content-Type: application/json

{
  "username": "johndoe",
  "email": "john@example.com",
  "password": "Password123!"
}
```

Response (201 Created):

```
{
  "message": "User registered successfully",
  "userId": "u987"
}
```

Error Responses:

- 400 Bad Request
 - Invalid email format
 - Username not alphanumeric (3-30 chars)
 - Password missing uppercase/number/special character
 - 409 Conflict
 - Email/username already exists
-

2. Login - Step 1: Request OTP

Send OTP to registered email

```
POST /auth/login/request-otp HTTP/1.1
Host: api.socialsphere.com
Content-Type: application/json

{
  "email": "john@example.com"
}
```

```
}
```

Response (200 OK):

```
{
  "message": "OTP sent to your email",
  "otpSessionId": "s12345"
}
```

Error Responses:

- 404 Not Found: Email not registered
 - 429 Too Many Requests: OTP request rate limit exceeded
-

3. Login - Step 2: Verify OTP

Exchange OTP for JWT token

```
http
POST /auth/login/verify-otp HTTP/1.1
Host: api.socialsphere.com
Content-Type: application/json

{
  "email": "john@example.com",
  "otp": "123456",
  "otpSessionId": "s12345"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOi...",
  "expiresIn": 3600,
  "roles": ["user"]
}
```

Responses:

- 400 Bad Request: Missing email, otp, or otpSessionId
- 401 Unauthorized: Invalid/expired OTP
- 404 Not Found: otpSessionId not found

4. Send Friend Request

```
Request friendship with another user
http
POST /friends/request HTTP/1.1
Host: api.socialsphere.com
Authorization: Bearer eyJhbGci...
Content-Type: application/json

{
  "toUserId": "u123"
}
```

Response (200 OK):

```
{
  "message": "Friend request sent"
}
```

Error Responses:

- 400 Bad Request:
 - toUserId is invalid
 - Self-request or duplicate pending request
- 401 Unauthorized: Missing/invalid JWT
- 403 Forbidden: User lacks user role
- 404 Not Found: toUserId doesn't exist

5. Accept/Reject Friend Request

Respond to a pending request

```
POST /friends/respond HTTP/1.1
Host: api.socialsphere.com
Authorization: Bearer eyJhbGci...
Content-Type: application/json

{
  "requestId": "r456",
  "action": "accept"
}
```

Response (200 OK):

```
{
  "message": "Friend request accepted"
}
```

Error Responses:

- 400 Bad Request:
 - action not "accept" or "reject"
 - Request already processed
 - 401 Unauthorized: Missing/invalid JWT
 - 403 Forbidden:
 - User lacks user role
 - User isn't the recipient
 - 404 Not Found: requestId doesn't exist
-

6. Create Post

Publish a new post

```
http
POST /posts HTTP/1.1
Host: api.socialsphere.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ6bnV5IiwiaWF0IjoxNTE2MjM5MDIyfQ%3D%3D
Content-Type: application/json

{
  "content": "Hello world! #social"
}
```

Response (201 Created):

```
{
  "postId": "p789",
  "authorId": "u987",
  "content": "Hello world! #social",
  "createdAt": "2025-07-08T12:34:56Z"
}
```

Error Responses:

- 400 Bad Request:

- Content empty/whitespace-only
 - Exceeds 256 characters
 - 401 Unauthorized: Missing/invalid JWT
 - 403 Forbidden: User lacks user role
-

7. Like Post

Like/unlike a post

```
http
POST /posts/p789/like HTTP/1.1
Host: api.socialsphere.com
Authorization: Bearer eyJhbGci...
```

Response (200 OK):

```
{
  "message": "Post liked",
  "postId": "p789",
  "likesCount": 6
}
```

Error Responses:

- 401 Unauthorized: Missing/invalid JWT
 - 403 Forbidden: User lacks user role
 - 404 Not Found: postId doesn't exist
-

8. List Posts

Fetch posts (paginated)

```
GET /posts?userId=u987&limit=10&offset=0 HTTP/1.1
Host: api.socialsphere.com
Authorization: Bearer eyJhbGci...
```

Response (200 OK):

```
[
  {
    "postId": "p789",
    "authorId": "u987",
    "content": "Hello world!",
    "likes": 5,
    "isLikedByMe": true,
    "createdAt": "2025-07-08T12:34:56Z"
  }
]
```

Error Responses:

- 400 Bad Request:
 - limit > 50 or negative
 - offset negative
- 401 Unauthorized: Missing/invalid JWT
- 403 Forbidden:
 - User lacks user role
 - Attempt to access private posts

Validation Rules

Ensuring data integrity and security:

Field	Rules
username	Alphanumeric, 3-30 chars, no spaces/special chars
email	Valid format (e.g., user@domain.com), unique
password	8+ chars, 1 uppercase, 1 number, 1 special character
otp	6-digit numeric, valid for 5 minutes, single-use

content	1-256 chars, non-whitespace
action	Exactly "accept" or "reject" (case-sensitive)
IDs (toUserId, postId, etc.)	Valid UUID format, references existing resources
Query Params (limit, offset)	Non-negative integers, limit between 1-50

Security Practices

Our API prioritizes robust security:

- **HTTPS Enforcement:** All traffic is encrypted to protect data in transit.
 - **Password Hashing:** Passwords are securely stored as **bcrypt hashes** with salting.
 - **JWT Security:**
 - Signed with **RSA256** for integrity.
 - **Short-lived tokens** (**expiresIn=3600s**).
 - **Server-side blacklist** implemented on logout to immediately invalidate tokens.
 - **OTP Verification:** Time-bound (5 minutes) and single-use codes enhance security.
 - **Rate Limiting:** Applied to OTP and login endpoints to prevent brute-force attacks.
 - **Input Sanitization:** Prevents common vulnerabilities like XSS/SQL injection by stripping HTML and escaping special characters.
 - **CORS Policy:** Only whitelisted domains are permitted to interact with the API.
 - **Error Handling:** Generic messages are provided for security-sensitive errors to avoid leaking information.
-

Database Schema

Tables & Columns

All IDs are **UUIDs** (Universally Unique Identifiers).

Table	Columns
Users	id, username, email, passwordHash, role, createdAt, updatedAt
OTPSessions	id, userId, otpCode, expiresAt, isUsed, createdAt
FriendRequests	id, fromUserId, toUserId, status (pending/accepted/rejected), requestedAt, respondedAt
Posts	id, userId, content, createdAt, updatedAt
Likes	id, userId, postId, likedAt (Unique constraint: userId + postId)

Key Design Notes

- **Indexes:** All foreign keys (userId, postId) are indexed for optimal query performance.
 - **Friendships:** Derived implicitly from FriendRequests where status='accepted'.
 - **Scalability:** The schema is optimized for horizontal scaling, featuring minimal joins and the use of UUIDs.
-